Uploading Code Through Termux

K. Sai Sreevallabh - EE25BTECH11031
Shreyas Goud Burra - EE25BTECH11051

CONTENTS

I Introduction 1
II Overview 1
III Firmware Compilation Using PlatformIO 1
III-A Create a C++ code file 1
III-B Generate .hex file: 1
IV TCP-UART Transparent Bridge 1
IV-A Steps 1
V Uploading Using AVRDUDE: 2
VI The Timing Issue 2
VII Hardware Assisted Timing Expansion 2
VIII Serial Monitor 2
IX A Step Ahead: Using PlatformIO Only 3
IX-A Editing the platformio.ini File 3
IX-B Delay Script: 3
IX-C Uploading the Code: 3

I. INTRODUCTION

Uploading code to Arduino through phone can be done using applications like ArduinoDroid; however uploading solely through Termux is a slightly different because Termux does not have the permissions to directly access the USB port on mobile phones. Hence, this document explains a software workaround to upload code onto an Arduino directly through termux only.

II. OVERVIEW

Since Termux cannot directly access the physical USB port on a mobile phone, a native Android application "TCP-UART Transparent Bridge" is employed as a middleware translator, to bypass the strict android permissions.

III. FIRMWARE COMPILATION USING PLATFORMIO

PlatformIO is a professional, command-line driven embedded development toolchain that compiles standard C++ code into raw microcontroller firmware. Ensure that the device on which this is implemented has PlatformIO already installed.

A. Create a C++ code file

```
nvim src/main.cpp
```

Type the code that must be uploaded and save the file.

B. Generate .hex file:

```
pio run
```

The .hex file is now saved in the path

```
.pio/build/uno/firmware.hex
```

The next section explain the working of the core of this approach, TCP-UART Bridge.

IV. TCP-UART TRANSPARENT BRIDGE

TCP-UART Transparent Bridge is an app (can be downloaded from Google PlayStore) which acts as a middleware, bridging the gap between Termux and the physical USB port.
Since it is a native graphical application, the Android OS grants it the proper, user-approved permissions to communicate directly with the Arduino's serial converter.
Once USB connection is claimed, the app opens a local Transmission Control Protocol (TCP) network port on the device (e.g. 192.168.0.XXX:8080).
Any data that is received over this local network is captured by this app and is injected to the physical USB port.
Similarly, any serial data from the Arduino is pushed through the local network, making it possible to replicate the Serial Monitor also.

A. Steps

- Once the application is installed from PlayStore, connect the Arduino board to the device using an OTG and click on "connect".
- Grant the USB permissions upon receiving the pop-up.
- Select the "server" option, under TCP and click start.
- This shows an IP-Address, as shown in the image.
- The IP-Address and Port will be used while uploading the code.

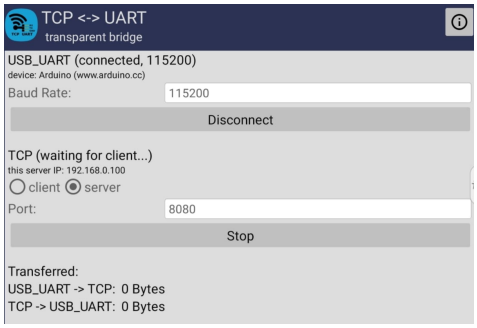

Fig. 1: Interface of TCP-UART Transparent Bridge App

V. Uploading Using AVRDUDE:

For installing AVRDUDE

```
apt install avrdude -y
```

- Traditionally, AVRDUDE expects to communicate with a physical hardware end point.
- However, due to restrictions on the Android device, AVRDUDE ability to transfer data over network sockets is leveraged.
- The single command to be used for uploading is listed below.

```
avrdude -p atmega328p -c arduino -P
    net:192.168.0.200:8080 -b 115200 -D -U
    flash:w:.pio/build/uno/firmware.hex
```

- However, the command must not be executed immediately. (This part is dealt with in the next section).
- The specified IP and Port in the command must perfectly match the server parameters configured in the TCP-UART app.
- By specifying the IPAddress, AVRDUDE treats the firmware upload as standard local network traffic.
- This network request is routed to the TCP-UART Android app, which receives the data packets and accurately injects them into the physical USB cable connected to the Arduino.

VI. The Timing Issue

- Uploading code onto the Arduino relies on an automatic reset mechanism.
- When a device opens a serial connection, it drops the DTR (Data Terminal Ready) line. This causes the a brief pulse on the ATMega328p's RESET pin.
- This immediately reboots it and forces it into the 'bootloader', which is a program that listens for a very short time to accept a new .hex file.
- While using the software workaround described above, the DTR line is not brought into the picture i.e. the microcontroller does not auto-reset to listen to the .hex file.
- The manual reset button on the Arduino must be pressed to reset the microcontroller, after which the code uploading command must be executed (i.e. pressing enter after typing the command)
- Since the bootloader window is very narrow, timing mismatch causes the AVRDUDE to timeout and upload to fail.

VII. Hardware Assisted Timing Expansion

- To solve the timing mismatch issue, an electrolytic capacitor ($100\mu F$ to $500\mu F$) is connected between the Arduino's Reset (RES) and Ground (GND) pins, with the negative lead grounded.
- The Arduino's reset line has an internal $10k\Omega$ pull-up resistor connected to the 5V supply. Inserting this capacitor creates a Resistor-Capacitor (RC) charging circuit.
- When the physical reset button is held down, the capacitor is instantly discharged, holding the microcontroller at 0V (active reset).
- When the button is released, the capacitor slowly charges through the internal pull-up resistor.
- The voltage at the reset pin over time is determined by the standard RC charging equation:

$$V_{reset}(t) = V_{supply}\left(1 - e^{\frac{-t}{RC}}\right)$$

.

- The microcontroller remains dormant in the reset state until the voltage crosses its internal logic-HIGH threshold (roughly 2.5V to 3.0V).
- This physical window (around 1 to 3 seconds) gives the user ample time to comfortably execute the code uploading command before the bootloader times out.

VIII. Serial Monitor

Once the code is successfully uploaded into the Arduino, Termux can also be used to view the serial data being sent from the Arduino to the device. This is acheived by using Netcat.

- Netcat is used to read and write arbitrary data across network connections using the TCP/IP protocol.
- It perfectly replaces the standard Serial Monitor by connecting directly to the TCP-UART Transparent Bridge app's local server and printing the incoming data to the terminal.
- Netcat can be installed using the command

```
apt install netcat-traditional -y
```

- To open Netcat and view serial data type the below command

```
nc 192.168.0.100 8080
```

- Unlike general Serial monitors, Netcat command does not require the baud rate to be specified because the connection between the terminal and TCP-UART Android app is a local network socket and operates at the maximum network speed.

- The actual hardware baud rate is managed by the TCP-UART Android application.

IX. A Step Ahead: Using PlatformIO Only

- In the above method, PlatformIO was used to compile the .hex file, and a separate AVRDUDE command was entered to upload the code, after keeping in mind the timing issues.
- PlatformIO has a native command to directly upload the code i.e. compile the C++ file into a .hex file and upload it to the microcontroller.
- The standard command prompts PlatformIO to search for a physical USB port through which the file can be flashed on the Arduino. However, without access to the physical USB port, the software workaround discussed above is employed and the timing considerations are taken into account.
- For implementing this method, the platoformio.ini file must be altered in order to customise the way in which the uploading is done.

A. Editing the platformio.ini File

The platformio.ini file acts as the control panel for the uploading process, defining precisely how the code should be compiled and transmitted to the board.
By default, Platformio searches for a physical USB port through which the code can be uploaded. To prevent PlatformIO from searching for a physical USB port, add the below line to the platformio.ini file.

```
    upload_protocol = custom
```

- Since the default upload method is disabled, an alternate upload command must be specified in the platformio.ini file.
- This command must send the compiled .hex file over to the local network socket.
- Below is the line to add after the upload protocol in the platformio.ini file.

```
upload_command = avrdude -C
    ~/.platformio/packages/tool-avrdude/avrdude.conf
    -p atmega328p -c arduino -b 115200 -P
    net:192.168.0.102:8080 -D -U flash:w:$SOURCE:i
```

- The IPAddress and Port should be the ones obtained from the TCP-UART Transparent Bridge app.

Save the platformio.ini file. However, there is another minor edit left, which is covered in the next subsection.

B. Delay Script:

- In a standard USB connection, PlatformIO automatically toggles the DTR line to reboot the Arduino.
- However, while using the software workaround employing the TCP-UART app, this cannot be directly done.
- Hence, the manual reset button must be pressed and released, to reboot the Arduino, keeping in mind the timing considerations.
- The standard command to compile a C++ file and upload the code using PlatformIO outputs multiple lines of dependency checks, build environments, and flash sizing before triggering the actual upload.
- Since compilation times vary based on the codebase, the exact time at which the upload begins cannot be discerned.
- Hence, a delay script is introduced, which intentionally pauses PlatformIO's upload sequence and display's a visual countdown upto the time the upload begins.
- Create a delay_script.py file and enter the code provided in

```
      codes/delay_script.py
```

C. Uploading the Code:

- Open the platformio.ini file once more and add the below line

```
        extra_scripts = delayScript.py
```

- Extra scripts is a parameter in PlatformIO which is used to modify PlatformIO's default behaviour by executing custom Python scripts at defined pre-action or post-action stages of the build environment.
- Make sure that delayScript.py is in the same working directory as the platformio.ini file while using this command; else clearly specify the file path.
- Save and close the platformio.ini file.
- Execute the below command to compile and upload the code

```
        pio run -t upload
```

- To manually synchronize the upload, press and hold the manual reset button as soon as the countdown initiates, and release it at around the 2 second mark to allow the bootloader time to initialize before AVRDUDE connects.
- Once the upload using PlatformIO is completed, a 'SUCCESS' message is printed in the terminal.
- However, sometimes a 'FAILED' message is printed with an upload error message. In spite of this, if the Writing and Reading are completed upto 100%, the code has been uploaded completely.
- Serial Monitor can be opened in the same way as described in section VIII.